

Отчёт по лабораторной работе №8

Выполнила студентка НКАбд-02-25

Валерия Сергеевна Григорьева

Содержание

1	Цель работы	3
2	Порядок выполнения лабораторной работы	4
2.1	Реализация переходов в NASM	4
2.2	Обработка аргументов командной строки	11
3	Задание для самостоятельной работы	21
4	Вывод	25

1 Цель работы

Цель данной работы - приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки

2 Порядок выполнения лабораторной работы

2.1 Реализация переходов в NASM

Для начала работы я создала каталог для лабораторной работы №8, перешла в него и создала файл lab8-1.asm (Рисунок 2.1).

```
vsgrigoreva@fedora:~$ mkdir ~/work/arch--pc/lab08
vsgrigoreva@fedora:~$ cd ~/work/arch--pc/lab08
vsgrigoreva@fedora:~/work/arch--pc/lab08$ touch lab8-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab08$
```

Рисунок 2.1: Создание каталога и файла для работы

Затем я ввела в файл lab8-1.asm текст программы из листинга 1 (Рисунок 2.2).

```
;-----
; Программа вывода значений регистра 'ecx'
;-----
%include 'in_out.asm'
SECTION .data
    msg1 db 'Введите N: ',0h

SECTION .bss
    N: resb 10
```

```

SECTION .text

    global _start
_start:

; ----- Вывод сообщения 'Введите N: '
    mov eax,msg1
    call sprint

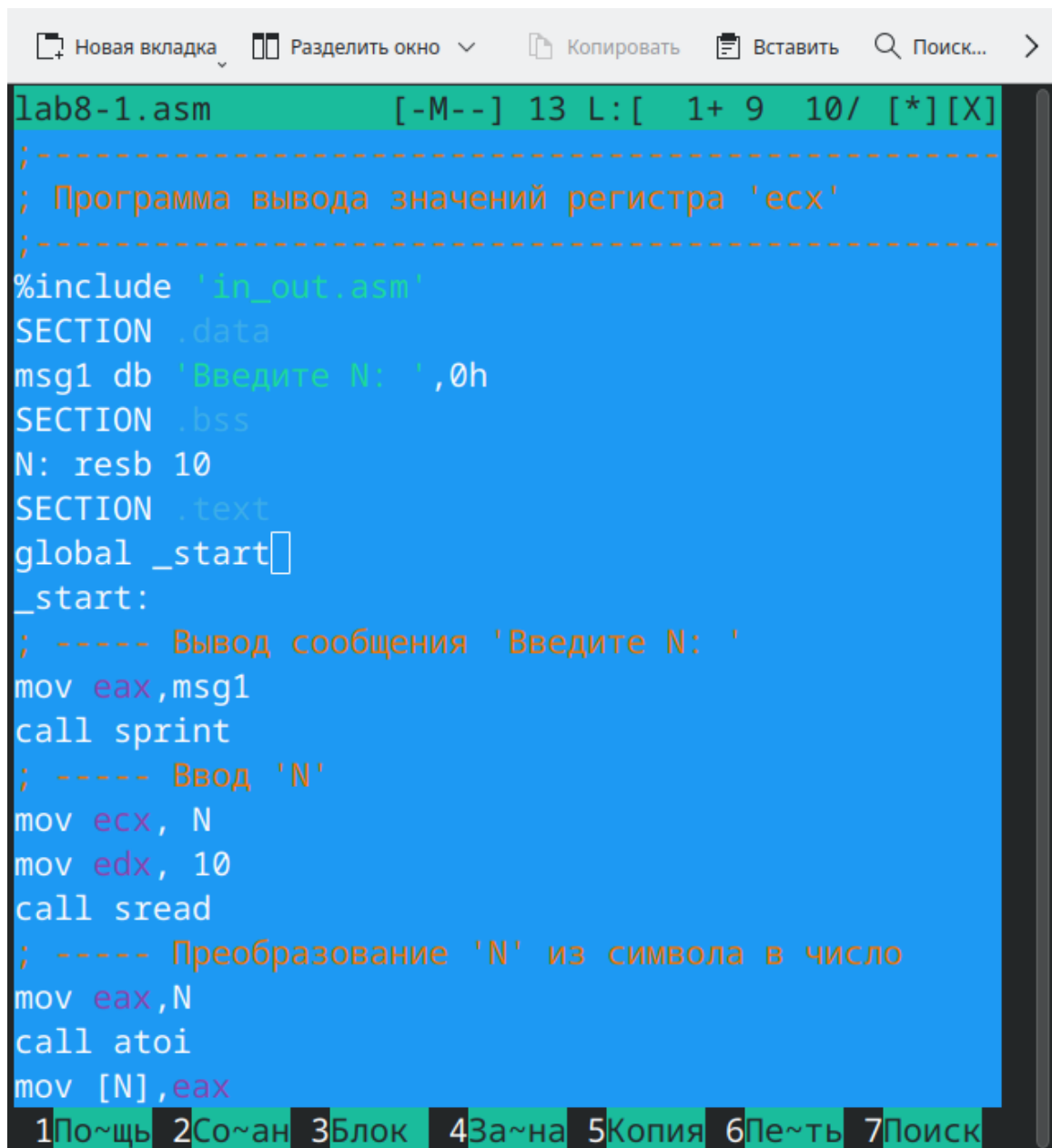
; ----- Ввод 'N'
    mov ecx, N
    mov edx, 10
    call sread

; ----- Преобразование 'N' из символа в число
    mov eax,N
    call atoi
    mov [N],eax

; ----- Организация цикла
    mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
    mov [N],ecx
    mov eax,[N]
    call iprintLF ; Вывод значения `N`
    loop label ; `ecx=ecx-1` и если `ecx` не '0'
                ; переход на `label`

call quit

```



```
lab8-1.asm      [-M--] 13 L:[ 1+ 9 10/ [*][X]
; -----
; Программа вывода значений регистра 'ecx'
; -----
#include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
1Почть 2Сочан 3Блок 4За~на 5Копия 6Пе~ть 7Поиск
```

Рисунок 2.2: Текст программы из файла lab8-1.asm

Затем я создала исполняемый файл и запустила его (Рисунок 2.3). Для корректной работы программы скопировала файл in_out.asm в каталог с файлом lab8-1.asm.

```

vsgrigoreva@fedora:~/work/arch--pc/lab08$ nasm -f elf lab8-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ./lab8-1
Введите N: 8
8
7
6
5
4
3
2
1
vsgrigoreva@fedora:~/work/arch--pc/lab08$ 

```

Рисунок 2.3: Создание исполняемого файла и запуск программы

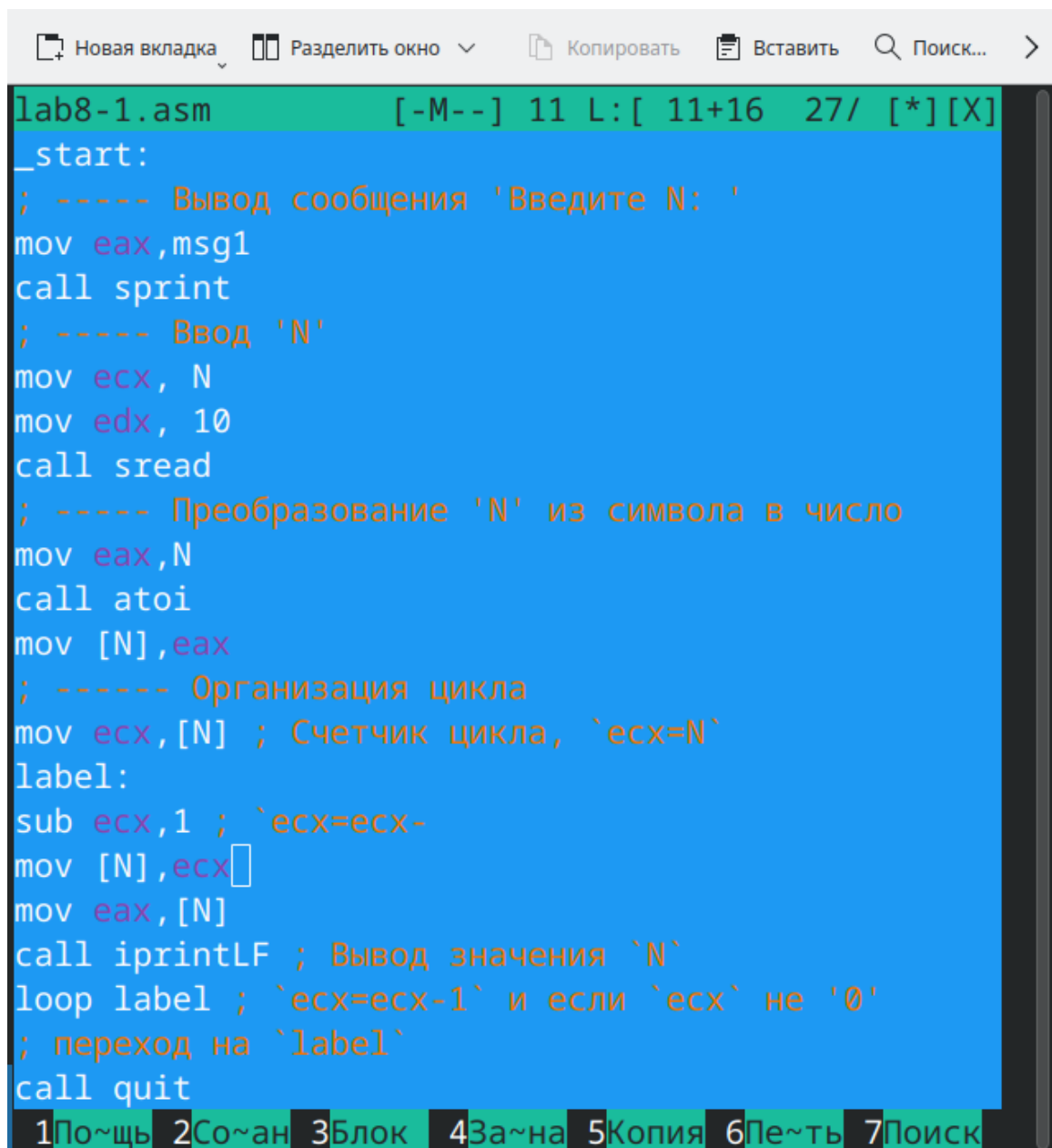
Данный пример показывает, что использование регистра `ecx` в теле цикла `loop` может привести к некорректной работе программы.

Далее я изменила текст программы, обавив изменение значение регистра `ecx` в цикле (Рисунок 2.4):

```

label:
    sub ecx, 1 ; `ecx=ecx-1`
    mov [N], ecx
    mov eax, [N]
    call iprintLF
    loop label

```



```
lab8-1.asm      [-M--] 11 L:[ 11+16 27/ [*][X]
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
sub ecx,1 ; `ecx=ecx-
mov [N],ecx
mov eax,[N]
call iprintLF ; Вывод значения `N`
loop label ; `ecx=ecx-1` и если `ecx` не `0`
; переход на `label`
call quit
1По~щ~ь 2Со~ан 3Блок 4За~на 5Копия 6Пе~ть 7Поиск
```

Рисунок 2.4: Измененный текст программы lab8-1.asm

Затем я создала исполняемый файл и запустила его (Рисунок 2.5).


```

vsgrigoreva@fedora:~/work/arch--pc/lab08$ nasm -f elf lab8-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ./lab8-1
Введите N: 8
7
5
3
1
vsgrigoreva@fedora:~/work/arch--pc/lab08$ 

```

Рисунок 2.5: Создание исполняемого файла и запуск программы

Регистр `ecx` принимает значения, которые уменьшаются дважды за одну итерацию. Число проходов не соответствует числу `N`.

Для использования регистра `ecx` в цикле и сохранения корректности работы программы можно использовать стек. Я внесла изменения в текст программы, добавив команды `push` и `pop` (добавления в стек и извлечения из стека) для сохранения значения счетчика цикла `loop` (Рисунок 2.6):

```

label:
    push ecx ; добавление значения ecx в стек
    sub ecx, 1
    mov [N], ecx
    mov eax, [N]
    call iprintLF
    pop ecx ; извлечение значения ecx из стека

    loop label

```

```

; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
push ecx ; добавление значения ecx в стек
sub ecx,1
mov [N],ecx
mov eax,[N]
call iprintLF
pop ecx ; извлечение значения ecx из стека
loop label
; переход на `label`
call quit

```

Рисунок 2.6: Измененный текст программы lab8-1.asm

Затем я создала исполняемый файл и запустила его (Рисунок 2.7).

```

vsgrigoreva@fedora:~/work/arch--pc/lab08$ nasm -f elf lab8-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ./lab8-1
Введите N: 8
7
6
5
4
3
2
1
0
vsgrigoreva@fedora:~/work/arch--pc/lab08$ 

```

Рисунок 2.7: Создание исполняемого файла и запуск программы

В данном случае число проходов цикла соответствует значению N, введенному с клавиатуры.

2.2 Обработка аргументов командной строки

Для начала работы я создала файл lab8-2.asm в каталоге ~/work/arch-pc/lab08 (Рисунок 2.8).

```
vsgrigoreva@fedora:~/work/arch-pc/lab08$ touch lab8-2.asm
vsgrigoreva@fedora:~/work/arch-pc/lab08$
```

Рисунок 2.8: Создание файла lab8-2.asm

Затем я ввела в него текст программы из листинга 8.2 (Рисунок 2.9).

```
;-----
; Обработка аргументов командной строки
;-----

%include 'in_out.asm'

SECTION .text

global _start

_start:

    pop ecx ; Извлекаем из стека в `ecx` количество
             ; аргументов (первое значение в стеке)

    pop edx ; Извлекаем из стека в `edx` имя программы
             ; (второе значение в стеке)

    sub ecx, 1 ; Уменьшаем `ecx` на 1 (количество
               ; аргументов без названия программы)

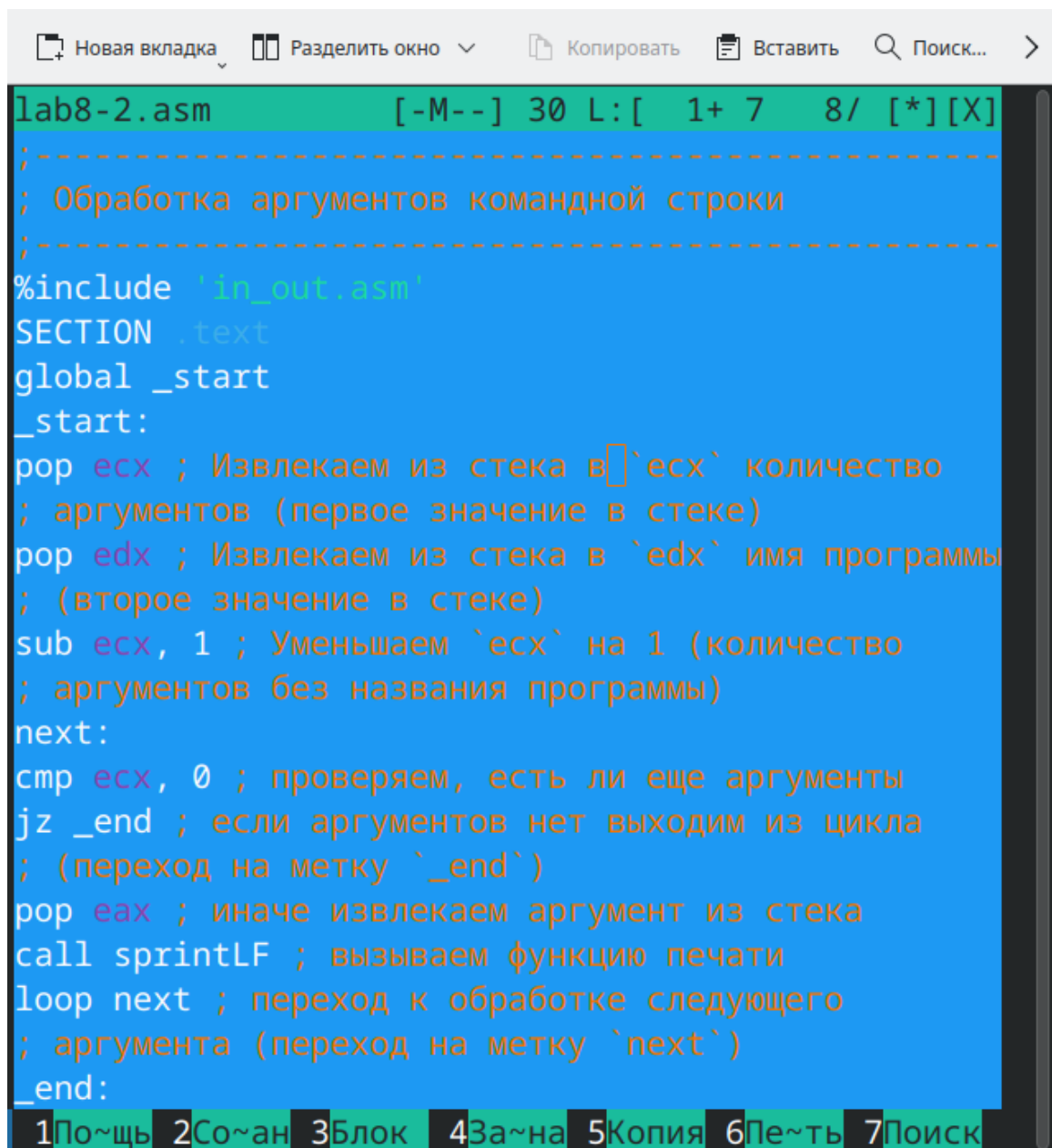
next:

    cmp ecx, 0 ; проверяем, есть ли еще аргументы
```

```
jz _end ; если аргументов нет выходим из цикла  
        ; (переход на метку `_end`)  
pop eax ; иначе извлекаем аргумент из стека  
call sprintf ; вызываем функцию печати  
loop next ; переход к обработке следующего  
        ; аргумента (переход на метку `next`)
```

_end:

```
call quit
```



```
lab8-2.asm      [-M--] 30 L:[ 1+ 7 8/ [*][X]
;-----
; Обработка аргументов командной строки
;-----
%include 'in_out.asm'
SECTION .text
global _start
_start:
pop ecx ; Извлекаем из стека в `ecx` количество
; аргументов (первое значение в стеке)
pop edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub ecx, 1 ; Уменьшаем `ecx` на 1 (количество
; аргументов без названия программы)
next:
cmp ecx, 0 ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
pop eax ; иначе извлекаем аргумент из стека
call sprintf ; вызываем функцию печати
loop next ; переход к обработке следующего
; аргумента (переход на метку `next`)
_end:
```

Рисунок 2.9: Текст программы lab8-2.asm

Затем я создала исполняемый файл и запустила его (Рисунок 2.10).

```

vsgrigoreva@fedora:~/work/arch--pc/lab08$ nasm -f elf lab8-2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ld -m elf_i386 -o lab8-2 lab8-2.o
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ./lab8-28 9 4
bash: ./lab8-28: Нет такого файла или каталога
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ./lab8-2 8 9 4
8
9
4
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ./lab8-2 8 9 '4'
8
9
4
vsgrigoreva@fedora:~/work/arch--pc/lab08$ 

```

Рисунок 2.10: Создание исполняемого файла и запуск программы

Программой было обработано 3 аргумента.

Затем я создала файл lab8-3.asm в каталоге ~/work/arch-pc/lab08 и ввела в него текст программы из листинга 8.3 (Рисунок 2.11).

```
%include 'in_out.asm'
```

```
SECTION .data
```

```
msg db "Результат: ",0
```

```
SECTION .text
```

```
global _start
```

```
_start:
```

```
    pop ecx ; Извлекаем из стека в `ecx` количество
                ; аргументов (первое значение в стеке)
```

```
    pop edx ; Извлекаем из стека в `edx` имя программы
                ; (второе значение в стеке)
```

```
    sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
                ; аргументов без названия программы)
```

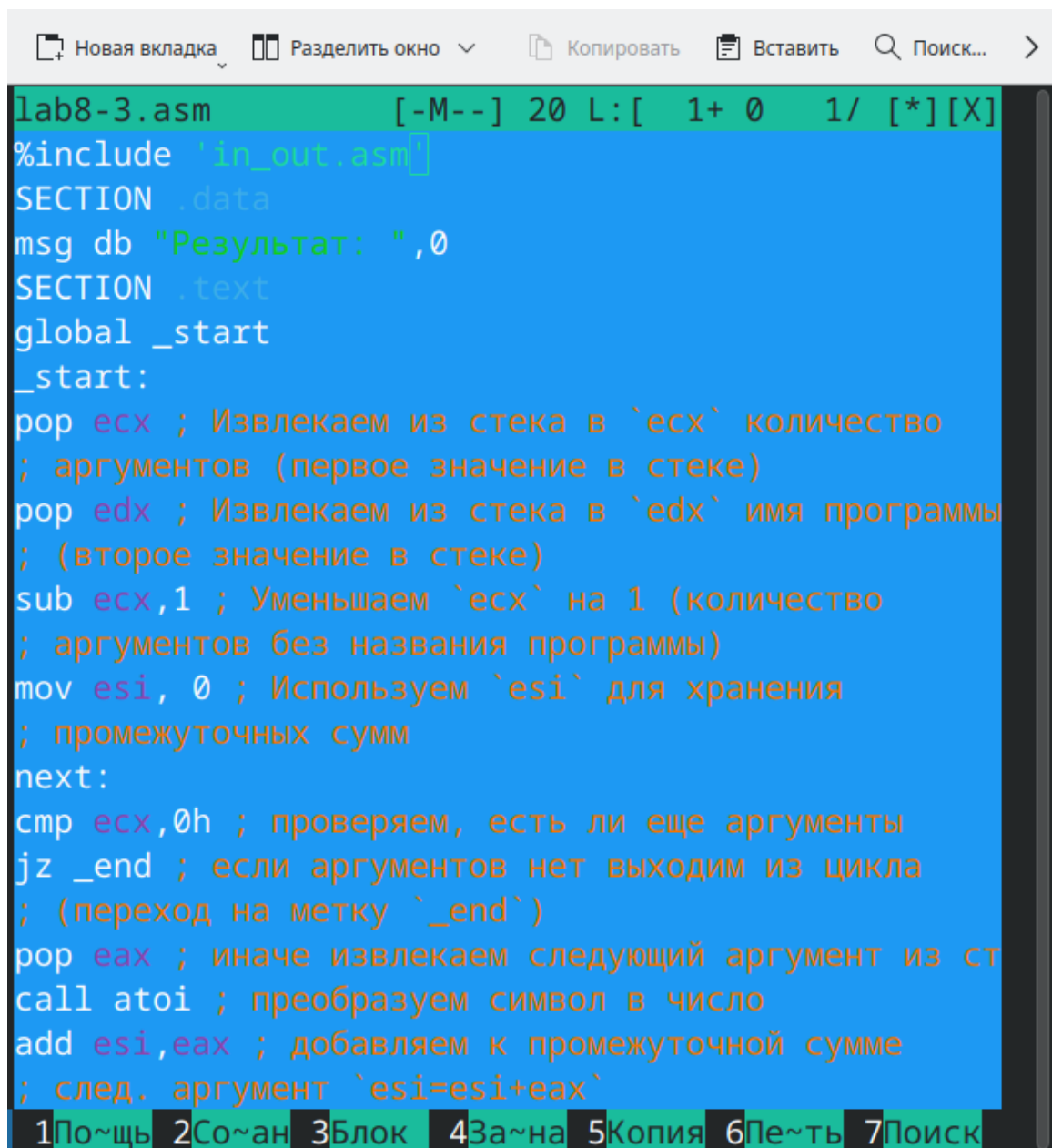
```
mov esi, 0 ; Используем `esi` для хранения  
; промежуточных сумм
```

next:

```
cmp ecx, 0h ; проверяем, есть ли еще аргументы  
jz _end ; если аргументов нет выходим из цикла  
; (переход на метку `_end`)  
pop eax ; иначе извлекаем следующий аргумент из стека  
call atoi ; преобразуем символ в число  
add esi, eax ; добавляем к промежуточной сумме  
; след. аргумент `esi=esi+eax`  
loop next ; переход к обработке следующего аргумента
```

_end:

```
mov eax, msg ; вывод сообщения "Результат: "  
call sprint  
mov eax, esi ; записываем сумму в регистр `eax`  
call iprintLF ; печать результата  
call quit ; завершение программы
```



```
lab8-3.asm [-M--] 20 L:[ 1+ 0 1/ [*][X]
#include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
pop ecx ; Извлекаем из стека в `ecx` количество
; аргументов (первое значение в стеке)
pop edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
; аргументов без названия программы)
mov esi, 0 ; Используем `esi` для хранения
; промежуточных сумм
next:
cmp ecx,0h ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
pop eax ; иначе извлекаем следующий аргумент из ст
call atoi ; преобразуем символ в число
add esi,eax ; добавляем к промежуточной сумме
; след. аргумент `esi=esi+eax`
1По~щ~ь 2Со~ан 3Блок 4За~на 5Копия 6Пе~ть 7Поиск
```

Рисунок 2.11: Текст программы lab8-3.asm

Затем я создала исполняемый файл и запустила его (Рисунок 2.12).


```

vsgrigoreva@fedora:~/work/arch--pc/lab08$ nasm -f elf lab8-3.asm
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ./lab8-3 12 45 96 45
Результат: 198
vsgrigoreva@fedora:~/work/arch--pc/lab08$ █

```

Рисунок 2.12: Создание исполняемого файла и запуск программы

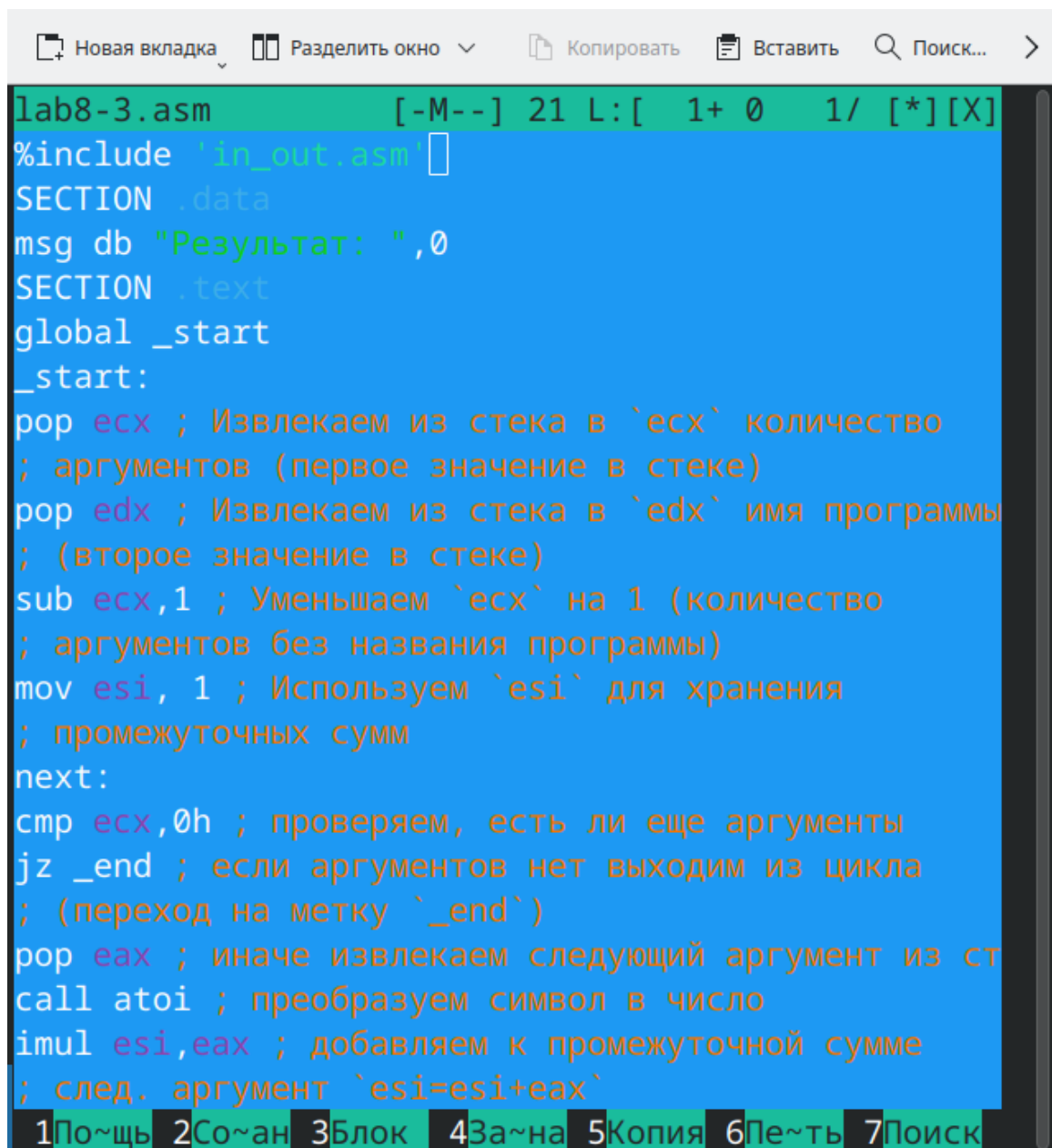
Далее я изменила текст программы из листинга 8.3 для вычисления произведения аргументов командной строки (Рисунок 2.13).

```

#include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
pop ecx ; Извлекаем из стека в `ecx` количество
; аргументов (первое значение в стеке)
pop edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
; аргументов без названия программы)
mov esi, 1 ; Используем `esi` для хранения
; промежуточных сумм
next:
cmp ecx,0h ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
pop eax ; иначе извлекаем следующий аргумент из стека
call atoi ; преобразуем символ в число
imul esi,eax ;

```

```
loop next ; переход к обработке следующего аргумента
_end:
mov eax, msg ; вывод сообщения "Результат: "
call sprint
mov eax, esi ; записываем сумму в регистр `eax`
call iprintLF ; печать результата
call quit ; завершение программы
```



```
lab8-3.asm [-M--] 21 L:[ 1+ 0 1/ [*][X]
#include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
pop ecx ; Извлекаем из стека в `ecx` количество
; аргументов (первое значение в стеке)
pop edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
; аргументов без названия программы)
mov esi, 1 ; Используем `esi` для хранения
; промежуточных сумм
next:
cmp ecx,0h ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
pop eax ; иначе извлекаем следующий аргумент из ст
call atoi ; преобразуем символ в число
imul esi,eax ; добавляем к промежуточной сумме
; след. аргумент `esi=esi+eax`
1По~щ~ь 2Со~ан 3Блок 4За~на 5Копия 6Пе~ть 7Поиск
```

Рисунок 2.13: Измененный текст программы lab8-3.asm

Затем я создала исполняемый файл и запустила его (Рисунок 2.14).

```
vsgrigoreva@fedora:~/work/arch--pc/lab08$ nasm -f elf lab8-3.asm
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ./lab8-3 12 45 96 45
Результат: 2332800
vsgrigoreva@fedora:~/work/arch--pc/lab08$
```

Рисунок 2.14: Создание исполняемого файла и запуск программы

3 Задание для самостоятельной работы

Я написала программу, которая находит сумму значений функции $f(x)$ для $x = 1, 2, \dots, n$, т.е. программа должна выводить значение $f(1) + f(2) + \dots + f(n)$. Значения n передаются как аргументы. Вид функции $f(x) = 6 \cdot x + 13$. (Рисунок 3.1)

```
%include 'in_out.asm'

SECTION .data
msg_func      db 'Функция: f(x)=6*x+13',0h
msg_res       db 'Результат: ',0h

SECTION .text
global _start
_start:
    ; Стек при входе (NASM): ... arg_n, ..., arg_1, argv0, argc
    pop ecx          ; ecx := argc
    pop edx          ; edx := argv0 (имя программы)
    sub ecx, 1        ; ecx := кол-во реальных аргументов (без имени программы)

    mov esi, 0        ; esi — накопитель суммы (32-bit)
```

```

    cmp ecx, 0
    jz .print_result

.loop_args:
    pop eax          ; eax := адрес очередного аргумента (строка)
    push ecx         ; сохранить ecx (atoi/подпрограммы могут изменить ecx)
    call atoi        ; преобразование строки (по адресу в eax) -> result в eax
                    ; теперь eax = числовое значение x
    pop ecx          ; восстановить ecx

    imul eax, 6      ; eax = eax * 6
    add eax, 13      ; eax = eax + 13 => eax = f(x)

    add esi, eax     ; esi += f(x)

    loop .loop_args ; dec ecx; если !=0 -> переход к .loop_args

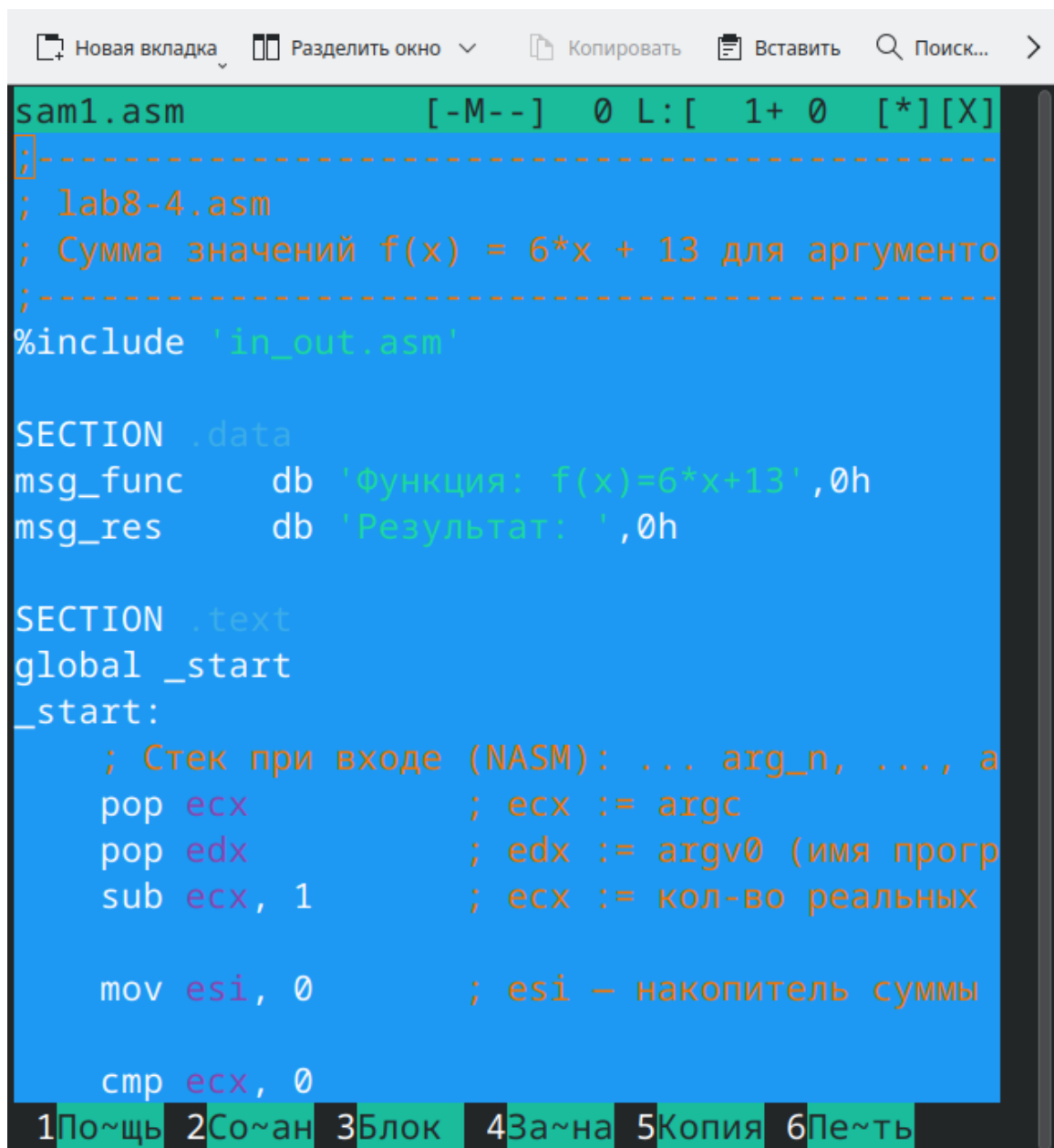
.print_result:
    mov eax, msg_func
    call sprintf

    mov eax, msg_res
    call sprintf

    mov eax, esi     ; результат в eax
    call iprintLF    ; печать результата с переводом строки

    call quit

```



```
sam1.asm [-M--] 0 L:[ 1+ 0 [*][X]
;-----
; lab8-4.asm
; Сумма значений  $f(x) = 6 \cdot x + 13$  для аргументов
;-----
%include 'in_out.asm'

SECTION .data
msg_func      db 'Функция:  $f(x)=6 \cdot x+13$ ',0h
msg_res       db 'Результат: ',0h

SECTION .text
global _start
_start:
    ; Стек при входе (NASM): ... arg_n, ..., а
    pop ecx      ; ecx := argc
    pop edx      ; edx := argv0 (имя програ
    sub ecx, 1    ; ecx := кол-во реальных

    mov esi, 0    ; esi — накопитель суммы

    cmp ecx, 0
```

1По~щ~ь 2Со~ан 3Блок 4За~на 5Копия 6Пе~ть

Рисунок 3.1: Текст программы

Затем я создала исполняемый файл и проверила его работу на нескольких наборах
= 1, 2, ..., (Рисунок 3.2).

```
vsgrigoreva@fedora:~/work/arch--pc/lab08$ nasm -f elf sam1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ld -m elf_i386 -o sam1 sam1.o
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ./sam1 1 2 3 4 5 6
Функция:  $f(x)=6*x+13$ 
Результат: 204
vsgrigoreva@fedora:~/work/arch--pc/lab08$ ./sam1 56 73 2
Функция:  $f(x)=6*x+13$ 
Результат: 825
vsgrigoreva@fedora:~/work/arch--pc/lab08$
```

Рисунок 3.2: Создание исполняемого файла и запуск программы

Программа корректно вывела значение и закончила работу.

4 Вывод

В ходе выполнения лабораторной работы я приобрела навыки написания программ с использованием циклов и обработкой аргументов командной строки.